

U

O

W

CSIT314

Software Development

Methodologies

Software process models



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Outline

- Software process models
 - Waterfall model
 - Agile methods (Scrum in particular)
 - Iterative/incremental model
 - Prototyping model
 - Spiral model



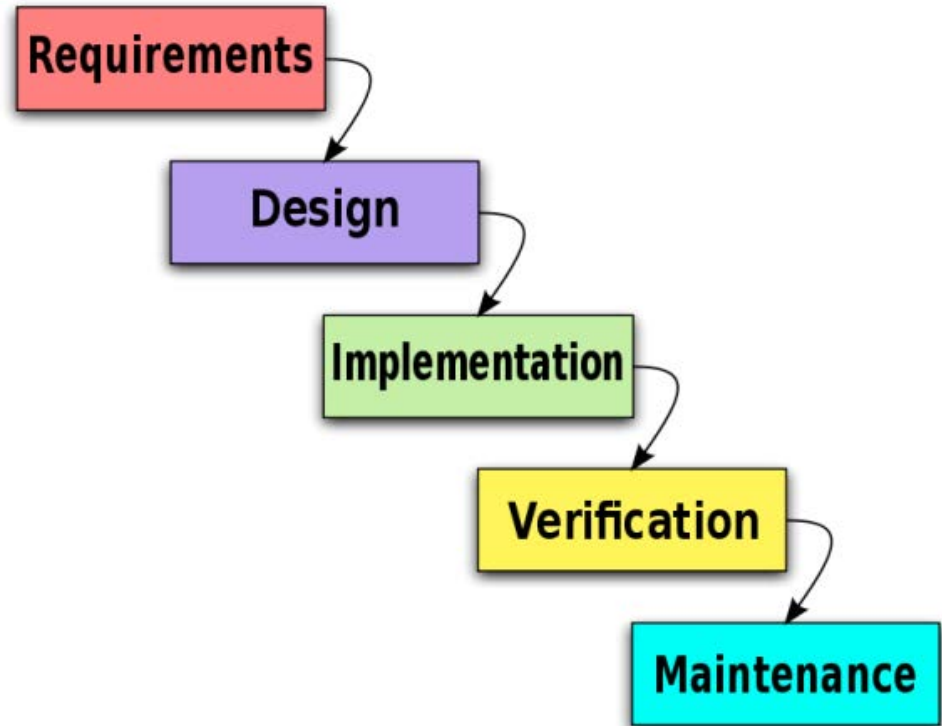
The software process

- A structured set of activities required to develop a software system, which involve:
 - Planning - feasibility, staff, identify risk, etc
 - Requirements analysis - defining what the system should do
 - Design & Implementation - defining how to build the system
 - Verification & Validation (V & V) - do the right thing in a right way
 - Maintenance and Evolution - changing the system in response to changing
- A software process model is an abstract representation of a process.



Waterfall model

- The waterfall model (adapted for software development since 1970s) is a sequential software development process.
 - flows from the top to the bottom
 - a phase has to be complete and absolutely correct before moving onto the next phase.



Waterfall model

- Waterfall project development process works well when:
 - Short term, small scope, less uncertainty
 - Requirements are very well understood and fixed at the outset of the project.
 - Product definition is stable and not subject to changes.
 - Technology is understood.
 - Ample resources with required expertise are available freely.
 - The project is of short duration.
- What happen if?
 - Customer tastes/ requirements changing
 - Disruptive technologies
 - Complex process
 - Such as producing a correct design before implementation and predicting problems
 - Long time frames



Agile methods

- In the 1990s, the IT industry faces substantial frustration
 - Most project followed the waterfall model
 - Big time gap between business requirements and the delivery of software to address those needs
 - Slow plan and/or design
 - Business environments and requirements quickly changed and the final product did not meet the then current needs.
- The waterfall model did not take advantage of how quickly software could be altered or developed



Agile methods

- Project planning
 - Emphasize on flexibility (resource constrain) and evolving customer requirements throughout the development process.
 - Allow flexibility react quickly to opportunities/risks
 - Planning all the time
 - Importance of evolving customer needs leads to incremental, iterative planning process
 - Stay connected with clients



Agile methods

- Agile refers to a set of methods and practices
 - requirements and solutions through the collaborative effort of teams and customers.
 - adaptive planning
 - iterative and incremental development
 - early delivery
 - continuous improvement
 - rapid and flexible response to change is encouraged



Agile principles

- Early and continuous delivery
- Welcome changing requirements
- Deliver working software frequently, with a preference to the shorter timescale
- Business people and developers must work together daily
- Support motivated individuals, and trust them to get the job done
- face-to-face conversation



Agile principles

- Working software (with specific functions)
- Sustainable development between developers and users to maintain a constant pace
- Attention to technical excellence and good design
- Simplicity--simplify their processes and elimination of unnecessary tasks, features--is essential
- Self-organizing teams, not relying on top-down management or extensive upfront planning
- The team learns to become more effective, then tunes and adjusts its behavior accordingly



Agile software development methods

- Scrum
- Extreme Programming
- Kanban
- Dynamic Systems Development Methods
- Lean software development
- Etc.

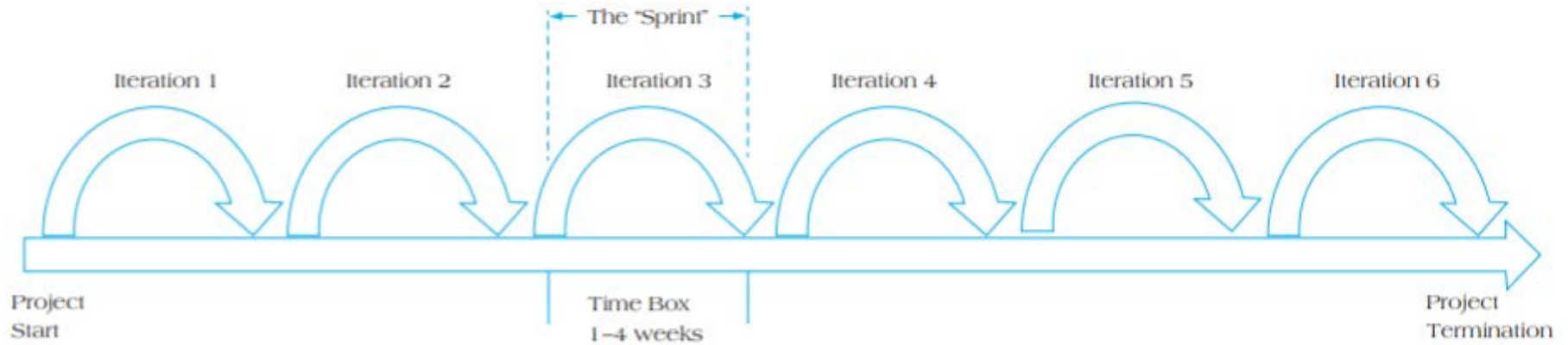


Scrum

- Flexible, iterative system designed for change and uncertainty
- Initial requirement => workable product => collect feedback
- “Rolling wave” process of continuous plan - execute - evaluate cycle
 - first come (request) first serve
 - recognise mistakes of assuming initial project
 - recognise the constraints (different from original specifications)
 - software projects are prone to constant changes
 - recognise customers’ requirement



Scrum



- Sprint—one iteration of the planning and executing cycle.
- Time-box—the length of any particular sprint, fixed in advance, during the Scrum meeting.



Scrum

- User stories—short explanation of the end user that captures what they do or what they need from the project under development (more later)
- Sprint backlog—the set of product backlog items selected for the Sprint, plus a plan for delivering the Sprint Goal. (from the develop team mainly)
- Burndown chart—remaining work in the Sprint backlog.
- Product backlog—a prioritized list of everything that might be needed in completed product and source of requirements for any changes (more later)
- Work backlog—evolving, prioritized queue of business and technical functionality that needs to be developed into a system



User stories

- User stories are short, simple descriptions of a feature told from the perspective of a user or customer
- User stories typically follow a simple template:
 - As a < type of user >, I want < some goal > so that < some reason >.
- Examples:
 - As a site visitor, I can read current news on the home page.
 - As a trainer, I can create a new course or event. This includes the following information: name, description (HTML), trainer names (multiple selection from a list), start date, end date, venue name (HTML) and address, contact name, contact phone, contact email, a link for more information, and a link to register. For a certification course the name of the class is a dropdown list; for others, it is free text.



Product backlog

- comprises the following different types of items:
 - Features
 - Bugs
 - Technical work (e.g. “Upgrade all developers’ workstations to Windows 11”)
 - Knowledge acquisition (e.g. “researching various JavaScript libraries and making a selection.”)
 - Prioritized by the product owner
 - Reprioritized at the start of each sprint



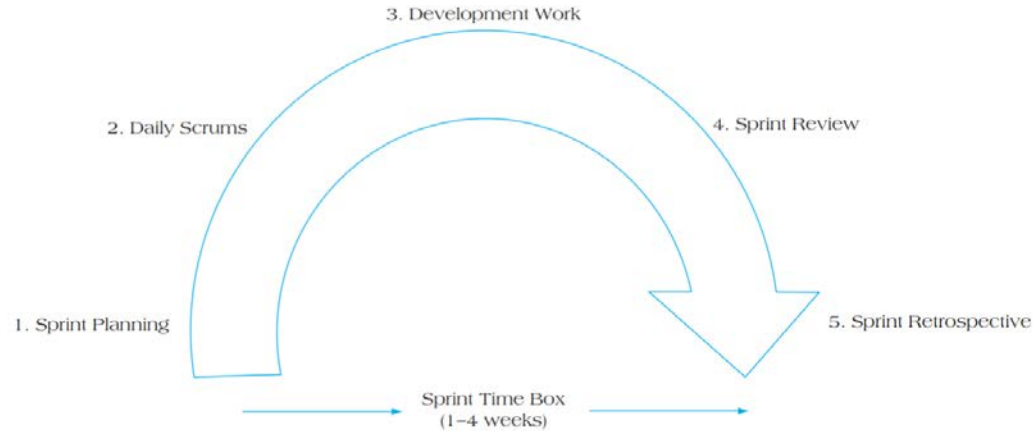
Scrum

- Product owner:
 - Define the features of the product
 - Decide on release date and content
 - Prioritize features according to market value
 - Adjust features and priority every iteration, as needed
 - Accept or reject work results
- Scrum Master
 - Represents management to the project, ensure that the team is fully functional and productive.
 - Responsible for enacting Scrum values and practices
- The team
 - Self-organizing and cross-functional: Programmers, testers, user experience designers, etc.



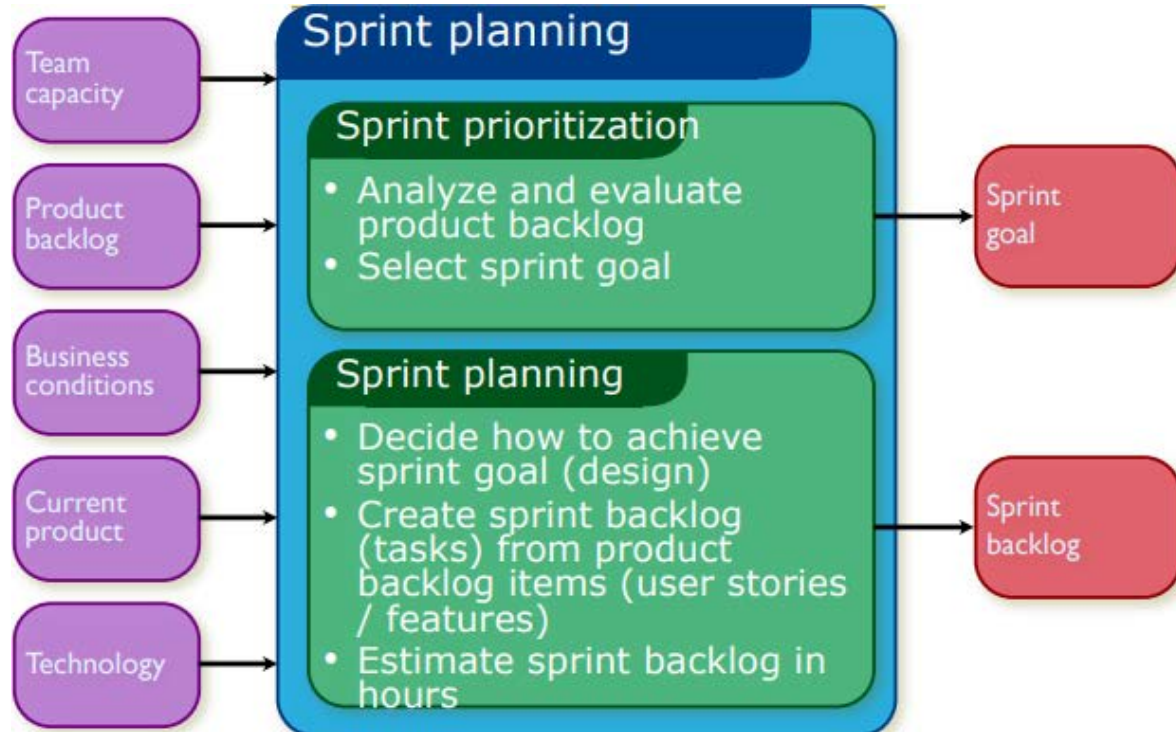
Steps

- Sprint Planning
- Daily Scrums
- Development Work
- Sprint Review
- Sprint Retrospective



Sprint Planning

- collaborative work of the entire Scrum team
- a maximum of one full day (eight hours) for a one-month Sprint
- Generate Sprint Backlog
 - What
 - How



Sprint planning

As a site visitor, I can read current news on the home page.

Develop UML design (4 hours)
Code the middle tier (8 hours)
Code the user interface (4)
Write test fixtures (4)
Code the foo class (6)

- Team decides on the goal for a particular sprint, and then selects items from the product backlog they commit to complete to achieve the sprint's goal.
- Sprint backlog is created
 - Tasks are identified and each is estimated (e.g. 1-16 hours)
 - Collaboratively, not done alone by the Scrum Master
 - High-level design is considered



Daily Scrum

- a short (15 minutes) event for information purposes
- synchronize their activities and create a plan for the next 24-hour time
- Leads to
 - Dev work
 - Sprint duration - 15 days
 - Team size - 5 members
 - Hours/day - 8
 - Total capacity - 600 hours

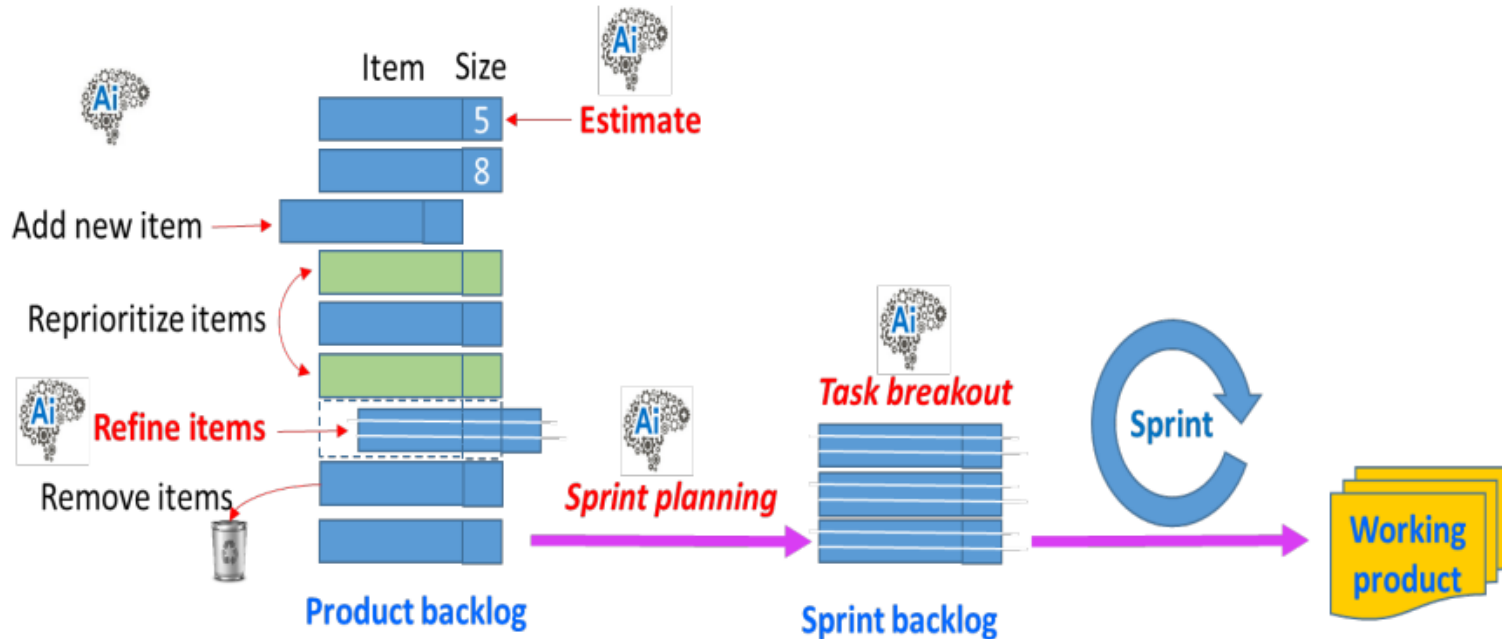


Sprint Review

- (informal) end of the Sprint to encourage feedback and collaboration
- the Scrum team and other key stakeholders
- leads to
 - Modified product backlog
 - Done vs not yet
 - impact
 - Next sprint backlog
 - Timeline, budget, etc
 - Problems, marketplace and/or potential uses



AI for Scrum Development

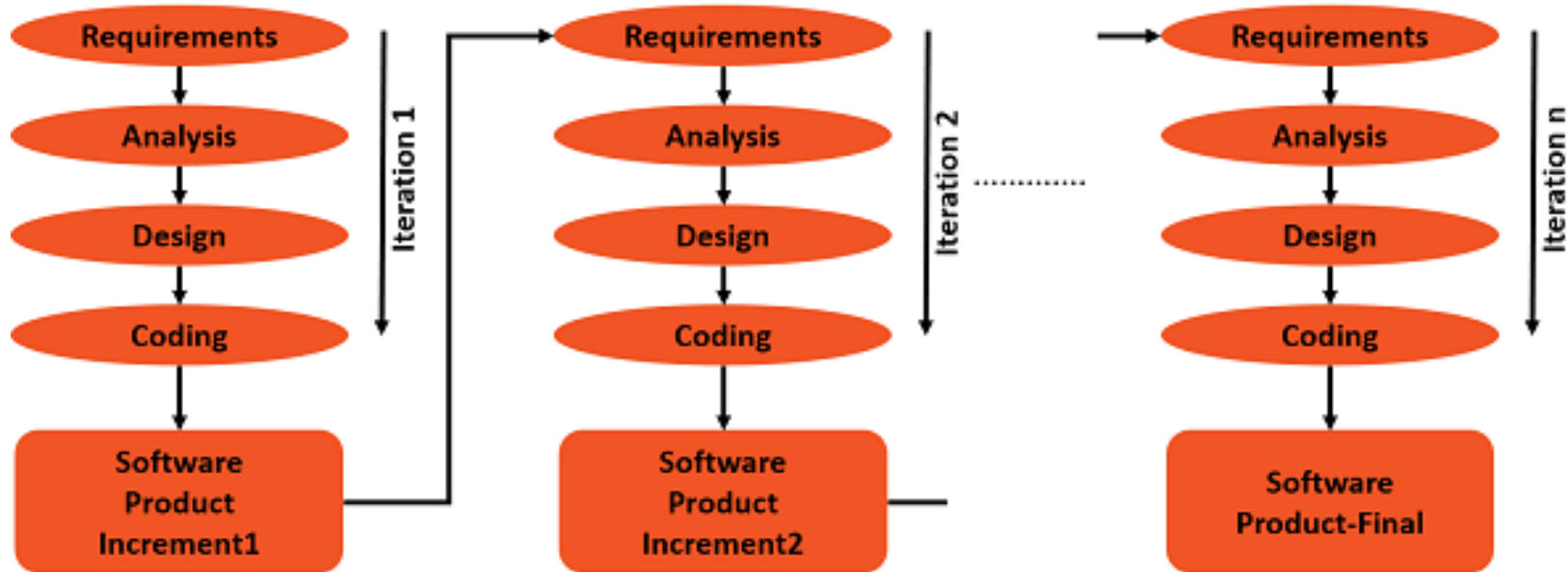


Incremental and Iterative model

- Repeated cycles (iterative)
 - User requirements are prioritised and the highest priority requirements are focus in early iteration
 - At each iteration, design modifications are made and new functional capabilities might be added
- Smaller portions (incremental)
 - Not a single delivery, but to break down
 - Each small one focuses on parts of required functionality



Incremental and Iterative model



Incremental and Iterative model benefits

- Feedback from early stages might still be used in developing latter stages
- The cost of accommodating requirement change is reduced
- More rapid delivery and deployment of useful software to the customer is possible.
 - Customers are able to use and gain value from the software earlier than is possible with a waterfall process



Incremental and Iterative model benefits

- Early increments act as a prototype to help improve requirements for later increments
- Lower risk of overall project failure (due to the break down)
- The highest priority system services tend to receive the most testing

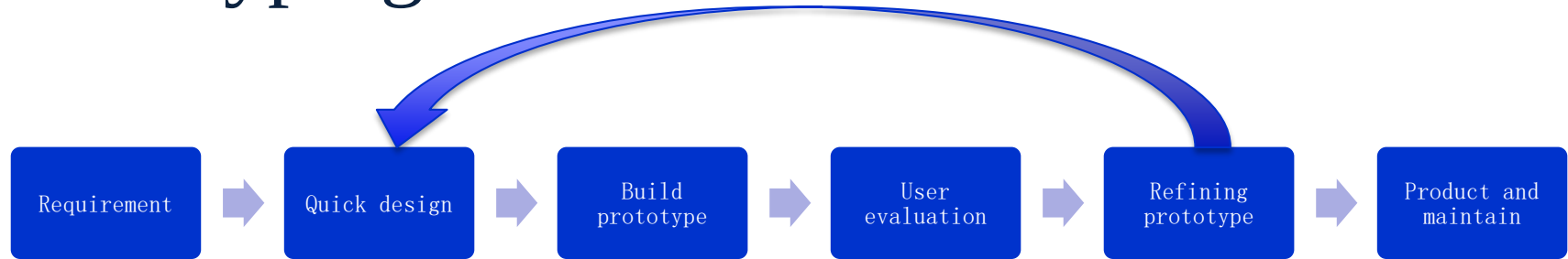


Possible disadvantages

- Loss of economy of scale
 - some costs will be repeated
- System structure tends to degrade as new increments are added.
 - regular change tends to corrupt its structure
 - refactoring is required (change to existing features)
 - incorporating further software changes becomes increasingly difficult and costly



Prototyping model



- gather requirements (user interview)
- quick design focusing on what will be visible to user - input & output formats
- build a prototype, i.e. a working version of the system
- prototype evaluated and requirements refined (user suggestion or comments)
- process iterated until customer & developer satisfied
 - then throw away prototype and “rebuild” system to high quality
- a key property to prototyping is the fact that it is supposed to be done quickly



Throw-away prototypes

- Prototypes should be discarded after development as they are not a good basis for a production system:
 - prototypes are normally undocumented
 - the prototype structure is usually degraded due to rapid change



Prototyping model benefits

- Users are actively involved in the development
 - exploring ideas and getting instant feedback from customers
- Reduced time and costs
 - a working model of the system is provided
 - the users get a better understanding of what developed
 - quicker user feedback
 - improve the quality of requirements and specifications provided to developers.
- Example projects to use prototyping
 - Human-Computer Interface



Prototyping model limitations

- User confusion of prototype and completed system
 - Users can begin to think that a prototype, intended to be thrown away, is the final system that can be delivered soon
 - Customers may not be willing to participate in a long-iteration cycle
- Developer attachment to prototype
 - Prototyping may encourage excessive change requests
 - Too many variations in software requirements



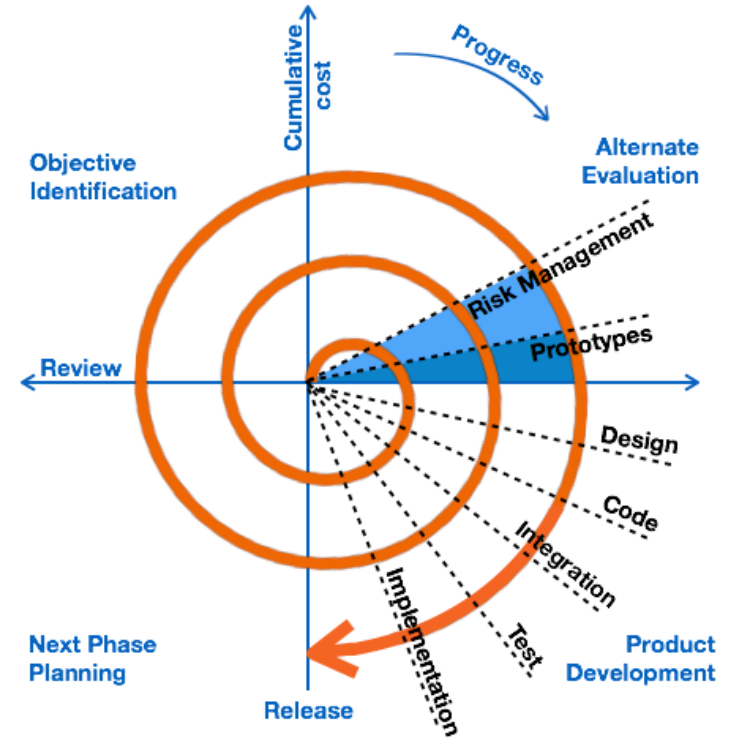
Prototyping model limitations

- Excessive development time of the prototype:
 - A key property to prototyping is the fact that it is supposed to be done quickly
 - the developers may try to develop a prototype that is too complex
- Expense of implementing prototyping:
 - A slow and time taking process, and to be thrown away
 - The costs for building a development team focused on prototyping may be high



Spiral Model

- A combination of waterfall with iterative model
 - Process is represented as a spiral rather than as a sequence of activities
- Workload is less at the beginning
- Allows incremental releases or refinement of the product through each iteration



Spiral Model

- helping people think about iteration in software processes and introducing the risk-driven approach
- mostly used in large projects:
 - Game development is a main area where the spiral model is used and needed
 - Military projects with a 2 year iteration



Limitations

- Complexity
 - multiple stages of planning, risk analysis, prototyping, and evaluation
 - requires extensive documentation at each stage of the process
 - iterative nature can lead to increased costs and time
 - may not be well-suited for small projects with well-defined requirements and limited scope
 - more beneficial for large-scale projects with complex requirements and significant uncertainties



Software Ethics

- Software ethics refers to the moral values and principles that guide the behavior and actions of individuals and organizations involved in software development.
 - Values: good things we strive for, desire and seek to protect.
 - Principles: outlining how we may or may not achieve our values (telling us what's right or wrong).
 - Purpose: reason for upholding our values and principles.



Why are ethics important in software development?

- Software are everywhere in people' s lives, across almost all industry sectors in our society
- Software can do good or cause harm
- Any software development activities (requirements, design, implementation, testing and maintenance) can lead to those consequences



ACM Code of Ethics

- Principle 1: PUBLIC – act consistently with the public interest, e.g. :
 - Accept full responsibility for their own work, and consider the public interest when they use our work.
 - Approve software only if they have a well-founded belief that it is safe, meets specifications, passes appropriate tests, and does not diminish quality of life, diminish privacy or harm the environment.
 - Disclose any actual or potential danger to the user, the public, or the environment.
 - Consider issues of physical disabilities, allocation of resources, economic disadvantage and other factors that can diminish access to the benefits of software.



ACM Code of Ethics

- Principle 2: CLIENT AND EMPLOYER – in the best interests of their client and employer, consistent with the public interest, e. g. :
 - Provide service in their areas of competence, being honest about any limitations of their experience and education.
 - Not knowingly use illegal or unethical software.
 - Keep private any confidential information gained in their professional work, where such confidentiality is consistent with the public interest and consistent with the law.



ACM Code of Ethics

- Principle 3: PRODUCT – ensure that their products and related modifications meet the highest professional standards possible, e.g.:
 - Strive for high quality, acceptable cost and a reasonable schedule, ensuring significant tradeoffs are clear to and accepted by the employer and the client, and are available for consideration by the user and the public.
 - Identify, define and address economic, cultural, legal and environmental issues related to work projects.
 - Ensure adequate testing, debugging, and review of software and related documents on which they work.



ACM Code of Ethics

- Principle 4: JUDGMENT – maintain integrity and independence in their professional judgment, e.g.:
 - Only endorse documents either prepared under their supervision or within their areas of competence and with which they are in agreement.
 - Maintain professional objectivity with respect to any software or related documents they are asked to evaluate.
 - Disclose to all concerned parties those conflicts of interest that cannot reasonably be avoided or escaped.



ACM Code of Ethics

- Principle 5: MANAGEMENT – subscribe to and promote an ethical approach to the management of software development and maintenance, e.g. :
 - Ensure good management for any project on which they work, including effective procedures for promotion of quality and reduction of risk.
 - Ensure that there is a fair agreement concerning ownership of any software, processes, research, writing, or other intellectual property to which a software engineer has contributed.



ACM Code of Ethics

- Principle 6: PROFESSION – advance the integrity and reputation of the profession consistent with the public interest, e.g. :
 - Take responsibility for detecting, correcting, and reporting errors in software and associated documents on which they work.



ACM Code of Ethics

- Principle 7: COLLEAGUES – be fair to and supportive of their colleagues, e.g.:
 - Credit fully the work of others and refrain from taking undue credit.
 - Review the work of others in an objective, candid, and properly-documented way.



ACM Code of Ethics

- Principle 8: SELF – shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession, e.g.:
 - Further their knowledge of developments in the analysis, specification, design, development, maintenance and testing of software and related documents, together with the management of the development process.
 - Improve their ability to create safe, reliable, and useful quality software at reasonable cost and within a reasonable time.

